Exercise Set 3

Problem 17

Task a

1. k-means algorithm is used for clustering problems, where the goal is to split the data points into k clusters based on their features.
2. Inputs are the number of clusters k and the dataset. The output is the assignment of each data point to one of the k clusters and the centroids of the clusters.
3. The results can be interpreted as each cluster representing a group of similar data points, with the centroid being the average position of all points in that cluster.

Task b

Cost of the k-means algorithm is $\mathcal{L} = \sum_{i=1}^{n} \|x_i - \mu_{c(i)}\|^2$ where c_i is the cluster assigned to point x_i and $\mu_{c(i)}$ is the centroid of that cluster.

During the assignment step, the cost $\mathcal{L}$ can only decrease or stay the same because each point is assigned to the nearest centroid, minimizing the distance for that point.

Task c

Calculation procedure at each step:

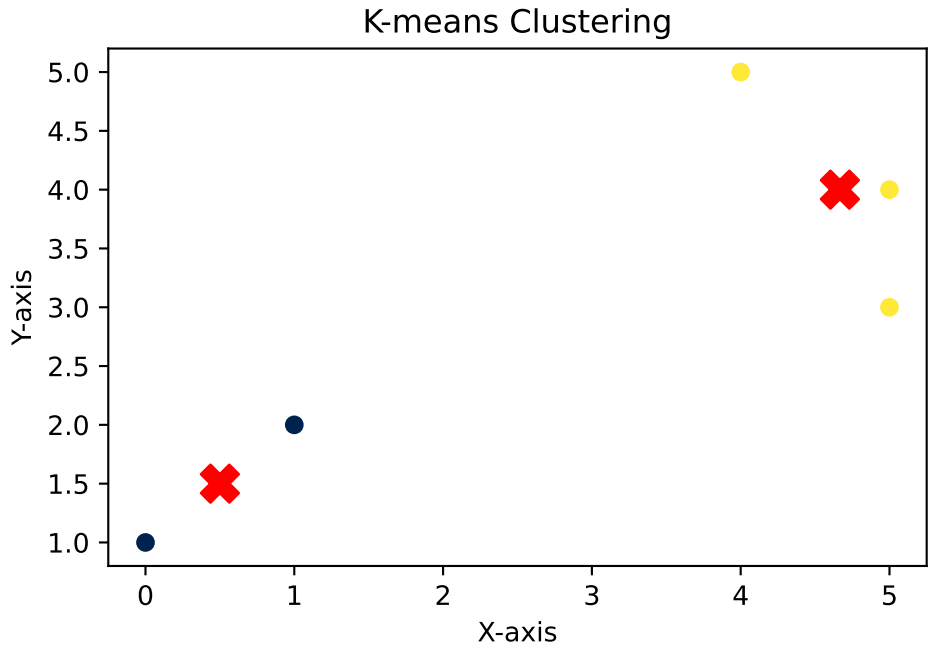
```
def assign_clusters(x, y, centroids):
    clusters = []
    for i in range(len(x)):
        distances = [float(np.sqrt((x[i] - centroids[j][0])**2 + (y[i] - centroids[j][1])**2)
        clusters.append(int(np.argmin(distances)))
        formatted_distances = [f"{d:.4g}" for d in distances]
        print (f"Point ({x[i]}, {y[i]}) distances to centroids: {formatted_distances}, assig
    print("------------")
```

```
    return clusters

def update_centroids(x, y, clusters, k):
    new_centroids = []
    for i in range(k):
        points = [(int(x[j]), int(y[j])) for j in range(len(x)) if clusters[j] == i]
        if points:
            new_centroids.append(np.mean(points, axis=0))
        else:
            new_centroids.append(centroids[i])
        formatted_centroid = [f"{x:.4g}" for x in new_centroids[-1]]
        print(f"Cluster {i} points: {points}, new centroid: {formatted_centroid}")
    print("============")
    return np.array(new_centroids)
```

```
Point (0, 1) distances to centroids: ['3.162', '4'], assigned to cluster 0
Point (1, 2) distances to centroids: ['2', '3.162'], assigned to cluster 0
Point (4, 5) distances to centroids: ['3.162', '4'], assigned to cluster 0
Point (5, 3) distances to centroids: ['4.123', '2.236'], assigned to cluster 1
Point (5, 4) distances to centroids: ['4', '3.162'], assigned to cluster 1
------------
Cluster 0 points: [(0, 1), (1, 2), (4, 5)], new centroid: ['1.667', '2.667']
Cluster 1 points: [(5, 3), (5, 4)], new centroid: ['5', '3.5']
============
Point (0, 1) distances to centroids: ['2.357', '5.59'], assigned to cluster 0
Point (1, 2) distances to centroids: ['0.9428', '4.272'], assigned to cluster 0
Point (4, 5) distances to centroids: ['3.3', '1.803'], assigned to cluster 1
Point (5, 3) distances to centroids: ['3.35', '0.5'], assigned to cluster 1
Point (5, 4) distances to centroids: ['3.59', '0.5'], assigned to cluster 1
------------
Cluster 0 points: [(0, 1), (1, 2)], new centroid: ['0.5', '1.5']
Cluster 1 points: [(4, 5), (5, 3), (5, 4)], new centroid: ['4.667', '4']
============
Point (0, 1) distances to centroids: ['0.7071', '5.548'], assigned to cluster 0
Point (1, 2) distances to centroids: ['0.7071', '4.177'], assigned to cluster 0
Point (4, 5) distances to centroids: ['4.95', '1.202'], assigned to cluster 1
Point (5, 3) distances to centroids: ['4.743', '1.054'], assigned to cluster 1
Point (5, 4) distances to centroids: ['5.148', '0.3333'], assigned to cluster 1
------------
Cluster 0 points: [(0, 1), (1, 2)], new centroid: ['0.5', '1.5']
Cluster 1 points: [(4, 5), (5, 3), (5, 4)], new centroid: ['4.667', '4']
============
```

```
Final clusters: [0, 0, 1, 1, 1]
Final centroids: [[0.5, 1.5], [4.666666666666667, 4.0]]
```



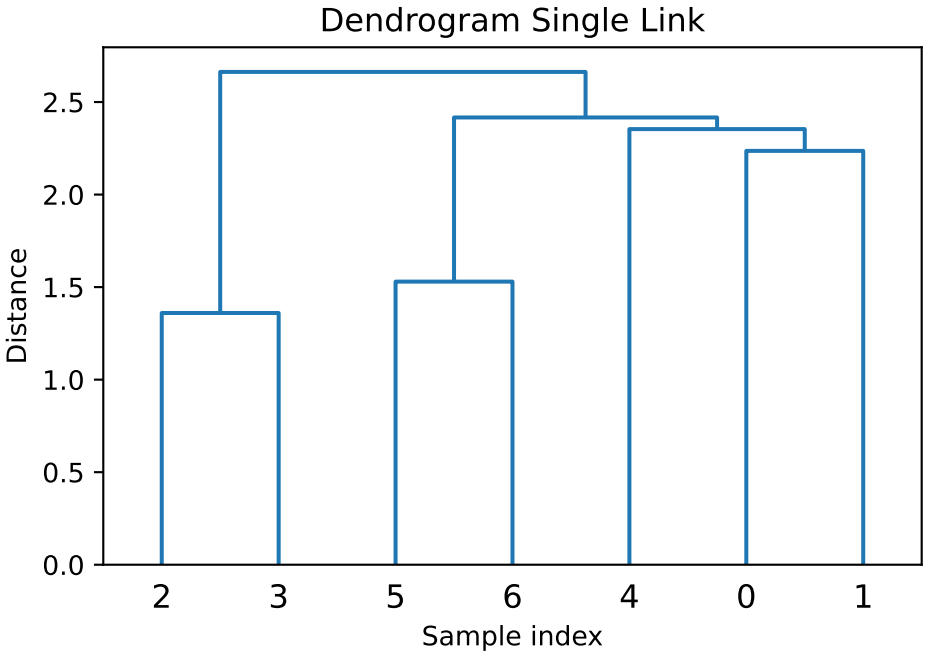
Problem 18

Task a and b

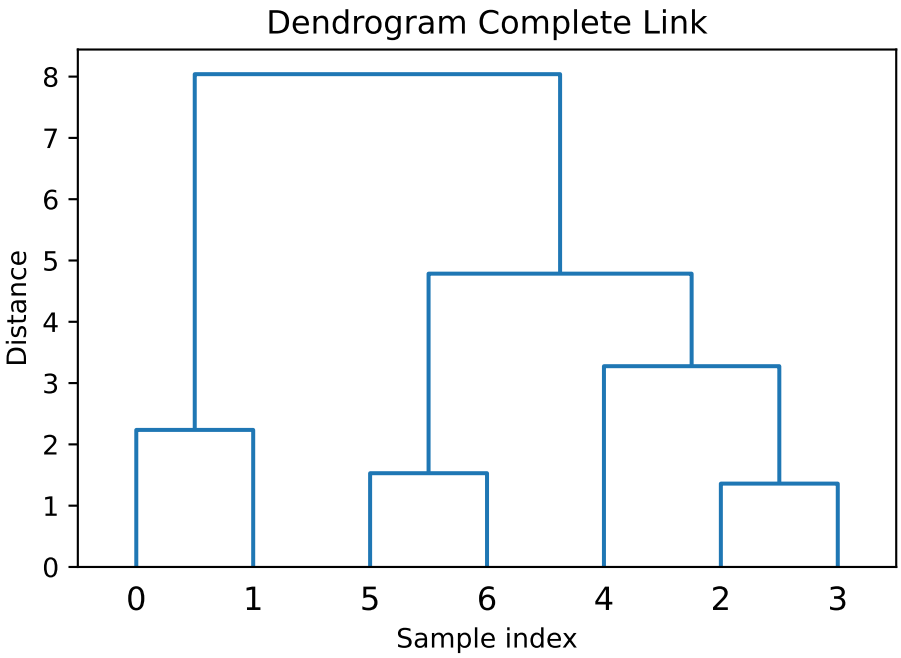
```
Pairwise Distance Matrix:
          0         1         2         3         4         5         6
0       0.0  2.236068  3.921734  5.281098  4.554119  6.519202    8.0399
1  2.236068       0.0   2.86007  4.036087   2.35372  4.527693  6.003332
2  3.921734   2.86007       0.0  1.360147  2.863564  3.328663  4.785394
3  5.281098  4.036087  1.360147       0.0  3.275668  2.662705  3.905125
4  4.554119   2.35372  2.863564  3.275668       0.0  2.416609  3.765634
5  6.519202  4.527693  3.328663  2.662705  2.416609       0.0  1.529706
6    8.0399  6.003332  4.785394  3.905125  3.765634  1.529706       0.0

Single Link Clustering:
Merging clusters containing points [2] and [3] with distance 1.36
Merging clusters containing points [5] and [6] with distance 1.53
Merging clusters containing points [0] and [1] with distance 2.236
```

```
Merging clusters containing points [0, 1] and [4] with distance 2.354
Merging clusters containing points [0, 1, 4] and [5, 6] with distance 2.417
Merging clusters containing points [0, 1, 4, 5, 6] and [2, 3] with distance 2.663
```

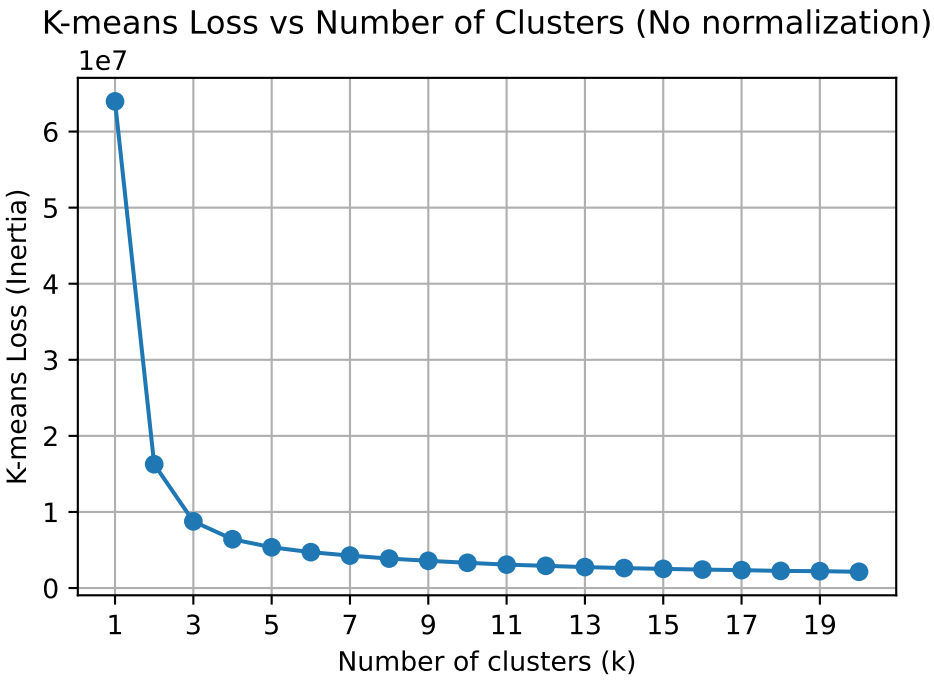


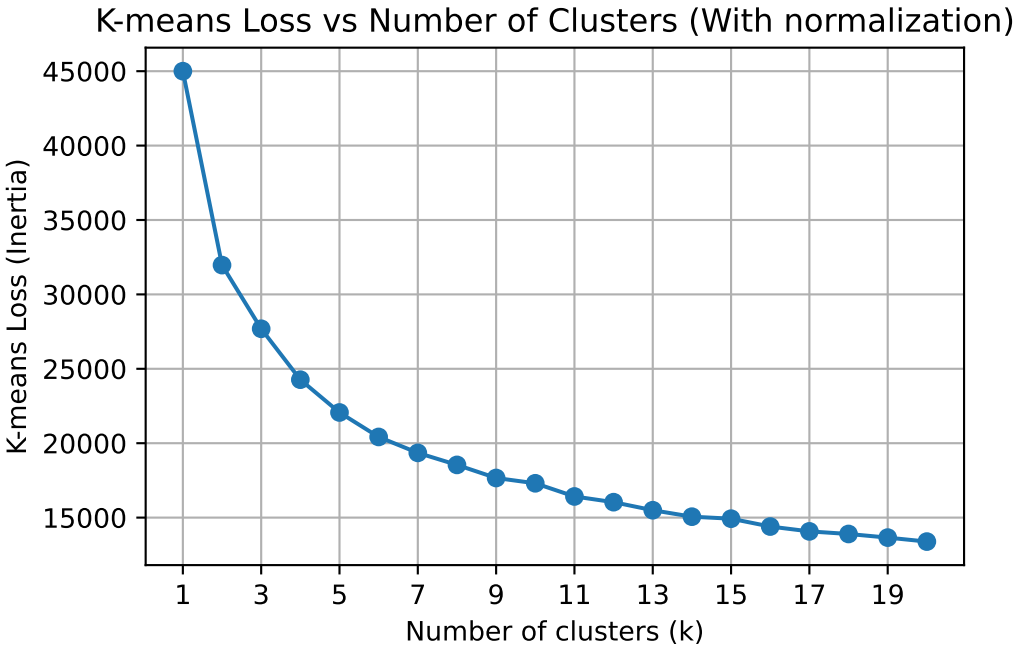

```
Complete Link Clustering:
Merging clusters containing points [2] and [3] with distance 1.36
Merging clusters containing points [5] and [6] with distance 1.53
Merging clusters containing points [0] and [1] with distance 2.236
Merging clusters containing points [2, 3] and [4] with distance 3.276
Merging clusters containing points [2, 3, 4] and [5, 6] with distance 4.785
Merging clusters containing points [0, 1] and [2, 3, 4, 5, 6] with distance 8.04
```


Dendrogram Complete Link
Distance
0
1
2
3
4
5
6
7
8
0
1
5
6
4
2
3
Sample index

Problem 19

Task a





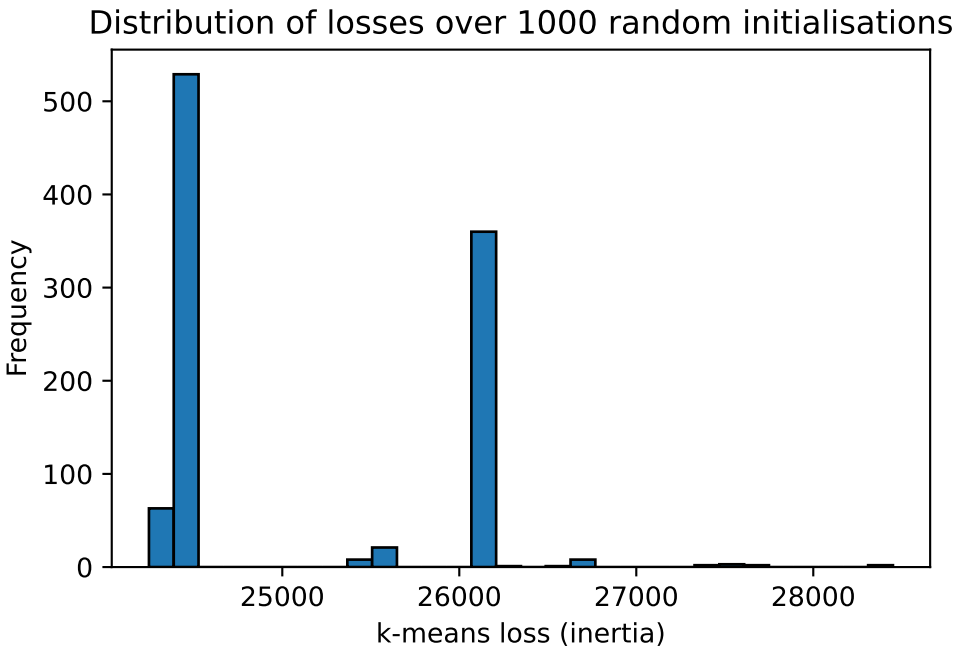
We can see the normalization significantly affects the k-means loss values. This happens because some values are significant;y larger in magnitude compared to others, which means the distance between them is larger as well. After normalization, all features contribute equally to the distance calculations, leading to a more balanced clustering outcome.

Task b

```
Confusion Matrix (after optimal assignment):
                cluster 0  cluster 1  cluster 2  cluster 3
class II               31          0         70         16
class Ia                0          0         21          5
class Ib               16          0         64          2
class nonevent         72         19         15        119
```

From confusion matrix we see that k-means can't cluster Ia correctly at all. Class II has a lot of erros as well as nonevent class. Class Ib is clustered best.

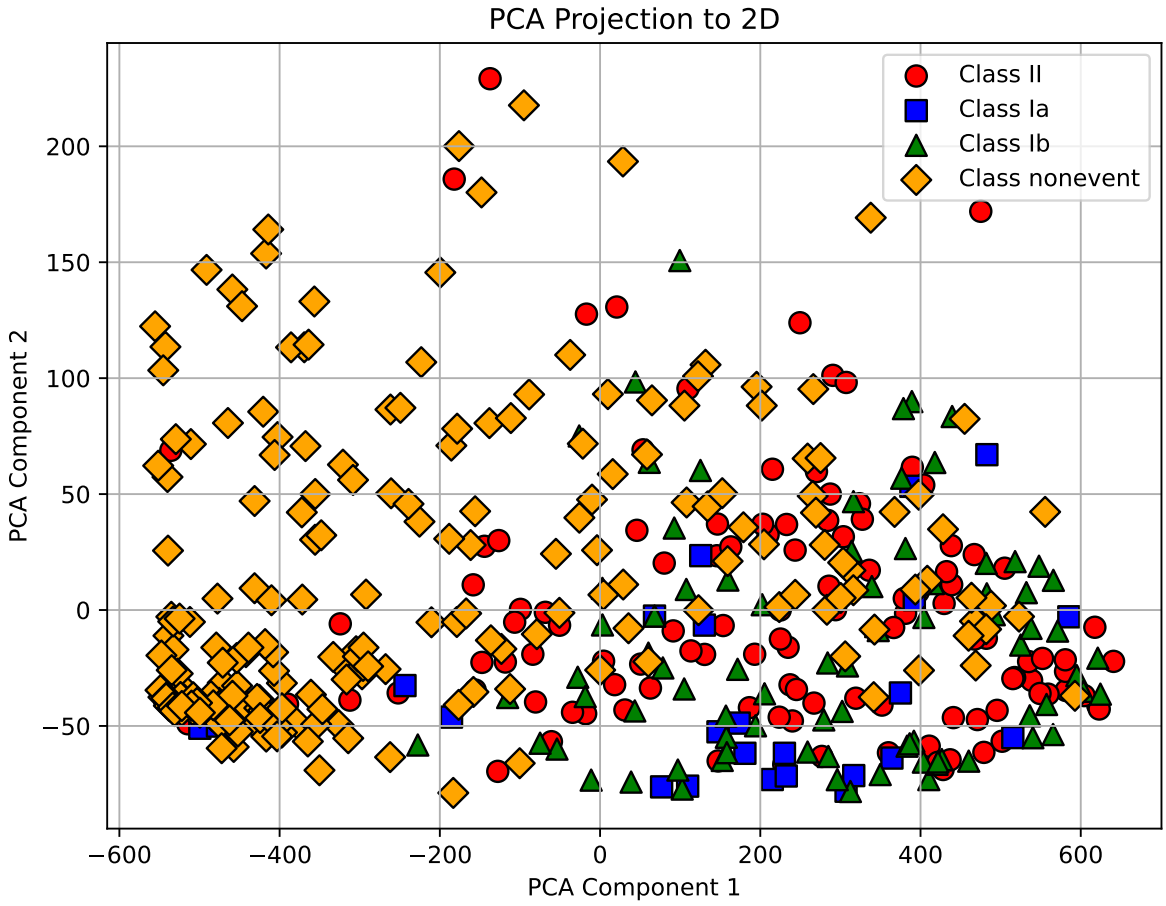
Task c



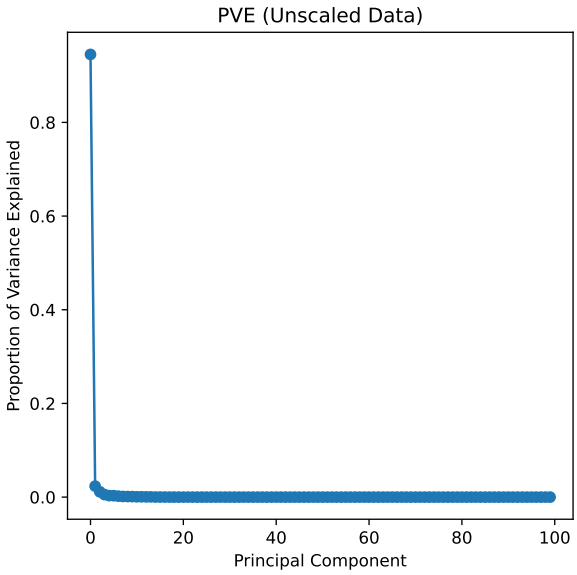

```
Best loss: 24246.571561892113
Worst loss: 28450.11226027048
Expected number of random initialisations needed: 1.7421602787456447
```

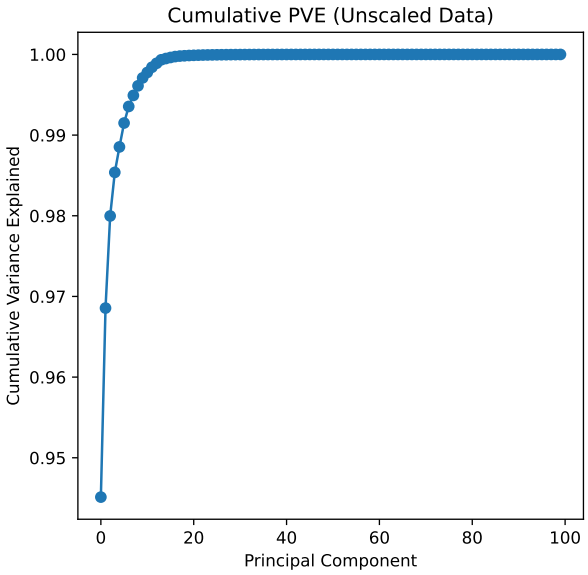
Problem 20

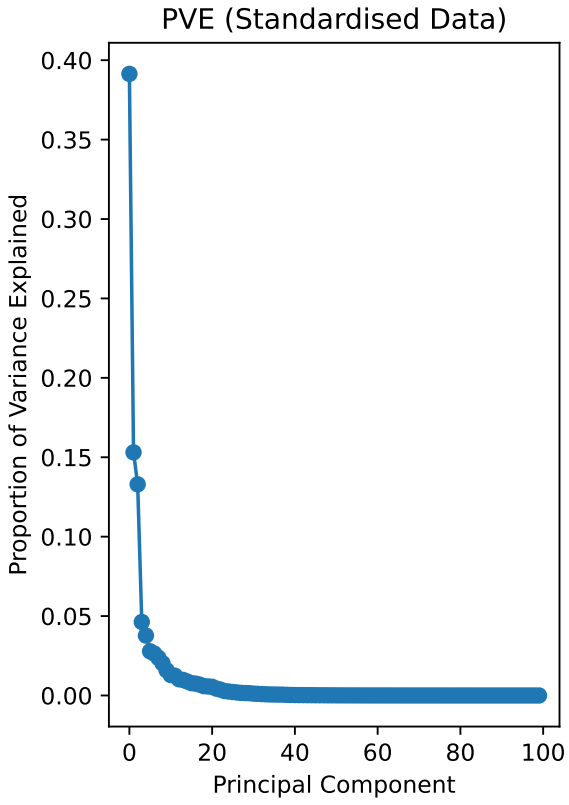
```
Explained variance ratio: [0.94512443 0.02343186]
```

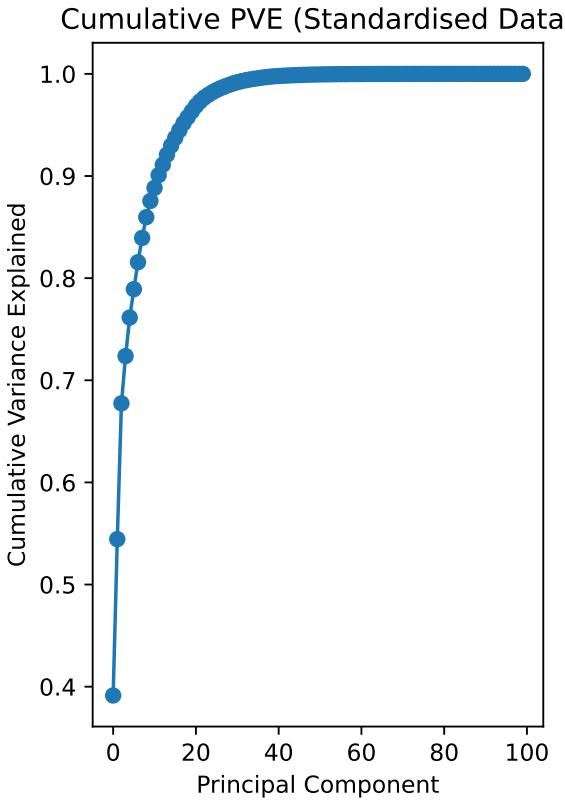


Task b


PVE (Unscaled Data)
Proportion of Variance Explained
0.0
0.2
0.4
0.6
0.8
0
20
40
60
80
100
Principal Component


Cumulative PVE (Unscaled Data)
Cumulative Variance Explained
0.95
0.96
0.97
0.98
0.99
1.00
0
20
40
60
80
100
Principal Component


PVE (Standardised Data)
Proportion of Variance Explained
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
0
20
40
60
80
100
Principal Component

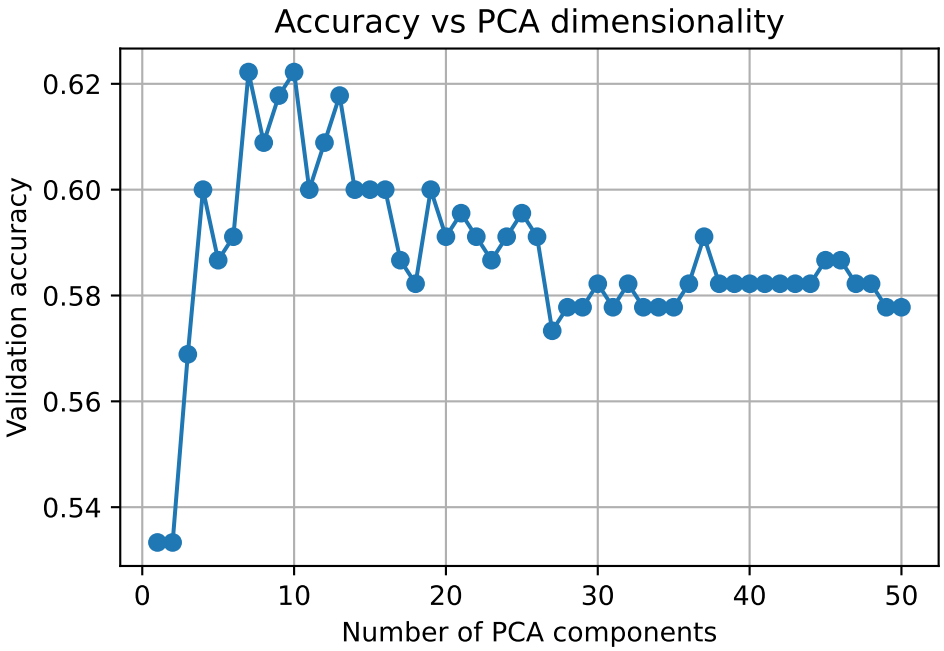

Cumulative PVE (Standardised Data)
Cumulative Variance Explained
0.4
0.5
0.6
0.7
0.8
0.9
1.0
0
20
40
60
80
100
Principal Component

Without normalization some features are extremely large in value compared to others, so PCA focuses on those features. After normalization, all features contribute equally to the variance, leading to a more balanced PCA result.

Task c

I decided to use k-NN classifier because it is simple and effective for multi-class classification problems. As the metric, I chose accuracy because it provides a straightforward measure of how well the classifier performs by calculating the proportion of correctly classified instances.

```
Accuracy without PCA: 0.6133
```




```
Maximum accuracy of 0.6222 achieved with 7 PCA components.
```

Problem 21

This weeks topics were quite new to me so I spent more time understanding them and doing the exercises than previous exercise set. PCA was the most confusing topic. As i want to go further in ML and AI in my work this whole course is perfect for me and i find it very useful. The topics also help with term project.